Life Sciences and
Facility Management

Institute of
Computational Life Sciences

Spatial filtering

Image and Signal Processing

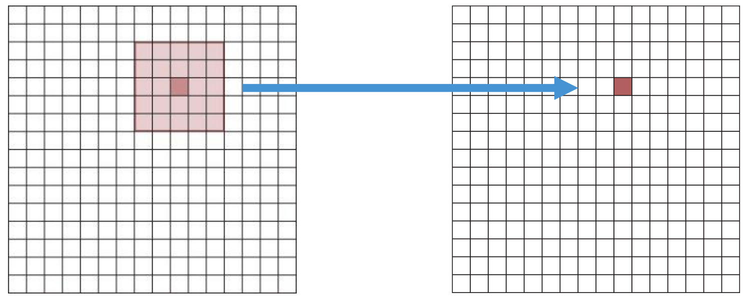
Norman Juchler

Spatial filtering

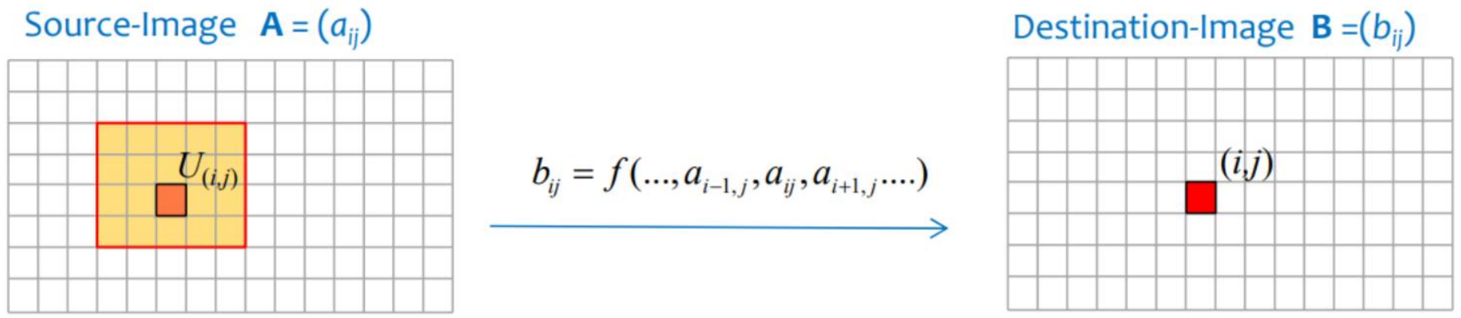
…using kernels

Idea

- **Local operations / local features**: Use the information around a pixel to compute a new value.



- Apply the same computation scheme to all pixels



Classification of local operations

Local operators can be classified according to the following criteria:

adaptive / not adaptive

adaptive filters are operators that are defined by multiple arithmetic rules which are used differently, depending on the data.

linear / not linear

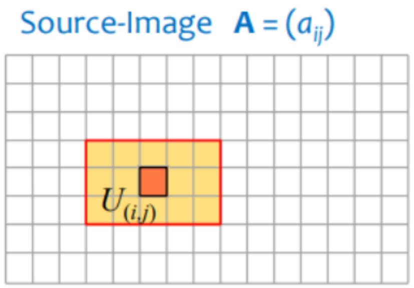
linear filters are operators whose computational rules can be described with a linear function

$$b = f(a_1, a_2, \ldots, a_n) = h_1 \cdot a_1 + h_2 \cdot a_2 + \ldots + h_n \cdot a_n$$

linear filters can be described by filter-masks!

Technical aspects of local operations

- Local operators are also referred to as filters (in the sense that certain image features are passed through, and others filtered out).
- The filter size (size of the environment) is an important parameter of the local operation. It determines the influence of the environment.
- For filters of odd size 3x3, 21x11, ... the operating point (i, j) is always in the center (otherwise it must be defined).

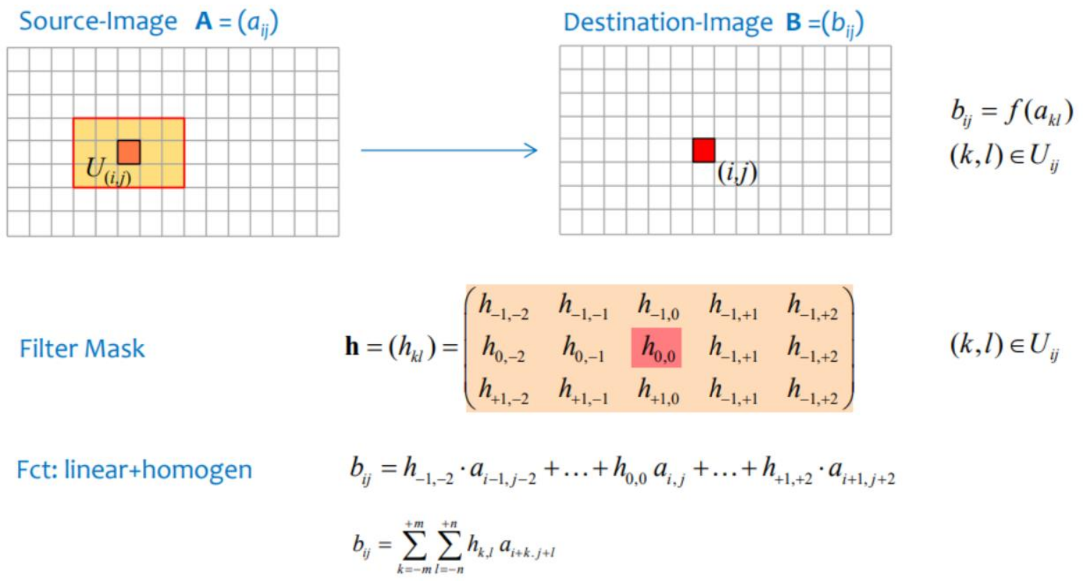


technical aspects

- The computation of the new pixel values at the image boundary must be treated specifically (e.g., reduction of the destination image, extension of the source image with zeros, ...)
- The calculated pixel values may not be written directly into the source image, otherwise a corruption in the calculation will occur (due to influence of neighboring pixels).

The kernel and the linear weighing scheme

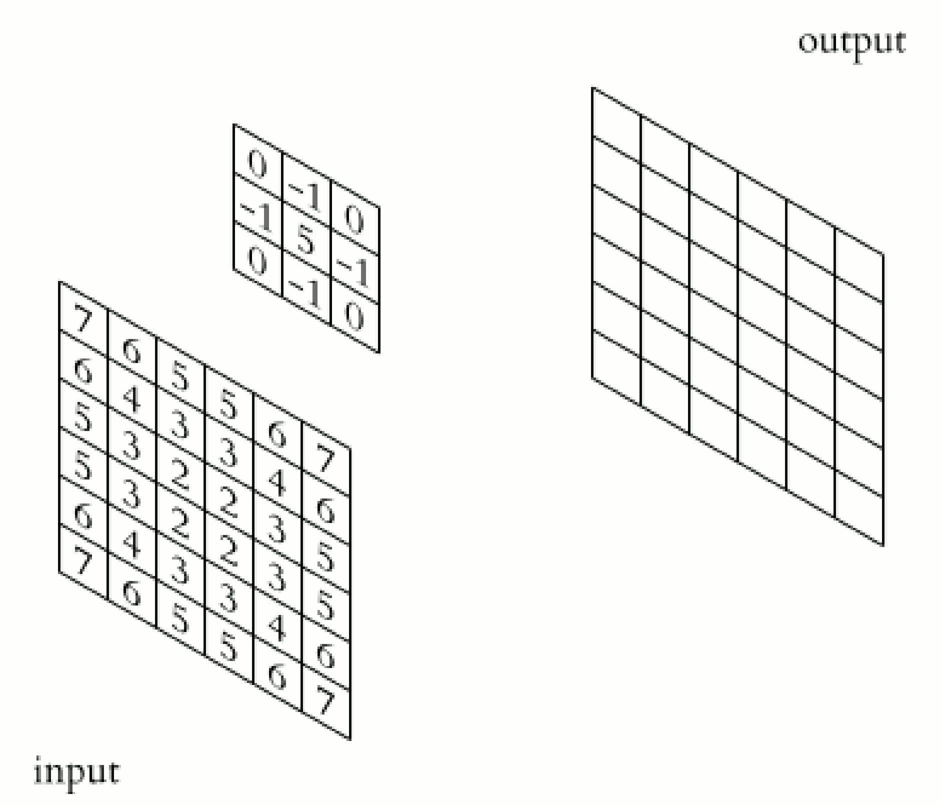
- Think of a kernel as the parameters of a simple weighing scheme for the pixels in the image A
- In case of a linear kernel, the application of a kernel only involves multiplications and additions:
  - Given a kernel h and the patch U around a pixel p
  - Operation 1: Elementwise multiplication of the kernel weights with the elements of the patch: $h \cdot U$
  - Operation 2: Summation



- Then sweep the kernel over all pixels.
- Mathematically, this is related to a convolution of kernel *h* with the input image *A*

The kernel and the linear weighing scheme

- Think of a kernel as the parameters of a simple weighing scheme for the pixels in the image A
- In case of a linear kernel, the application of a kernel only involves multiplications and additions:
  - Given a kernel h and the patch U around a pixel p
  - Operation 1: Elementwise multiplication of the kernel weights with the elements of the patch: $h \cdot U$
  - Operation 2: Summation
- Then sweep the kernel over all pixels.
- Mathematically, this is related to a convolution of kernel *h* with the input image *A*



The kernel and the linear weighing scheme

The linear filtering and the convolution are almost identical but not quite.

filtering $$(h \circ f)(x) = \int_{-\infty}^{\infty} h(s)\, f(x+s)\, ds \qquad \left(= \int_{-\infty}^{\infty} h(s-x)\, f(s)\, ds \right)$$

convolution $$(h * f)(x) = \int_{-\infty}^{\infty} h(s)\, f(x-s)\, ds \qquad \left(= \int_{-\infty}^{\infty} h(x-s)\, f(s)\, ds \right)$$

The convolution can be understood as a filtering with a mirrored filter function

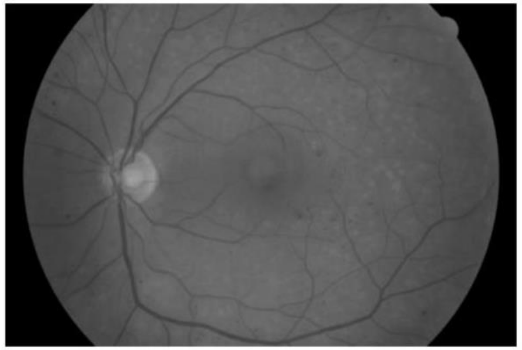
$$(h \circ f)(x) = (\tilde{h} * f) \qquad \Leftrightarrow \qquad \tilde{h}(x) := h(-x)$$

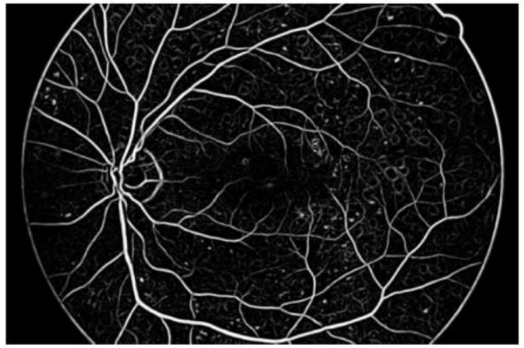
proof: $$(\tilde{h} * f)(x) = \int_{-\infty}^{\infty} \tilde{h}(x-s)\, f(s)\, ds = \int_{-\infty}^{\infty} \tilde{h}(-(s-x))\, f(s)\, ds = \int_{-\infty}^{\infty} h(s-x)\, f(s)\, ds = (h \circ f)(x)$$

The calculation rules that apply to the convolution can be used to process the filter masks.

Usages for local operations

- Reduce the amount of details in images
- Denoising
- Image sharpening
- Capture local features (like edges or corners)



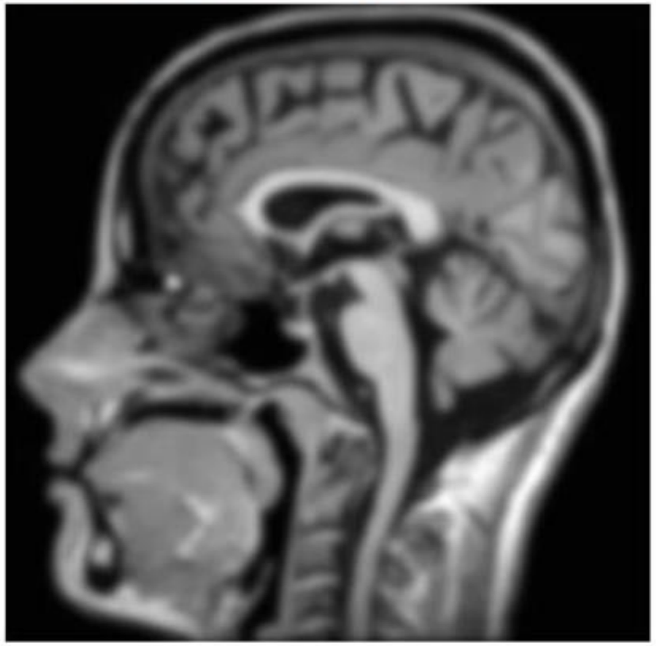


Smoothing / denoising

- Note: There are edge-preserving and non-edge preserving smoothing methods

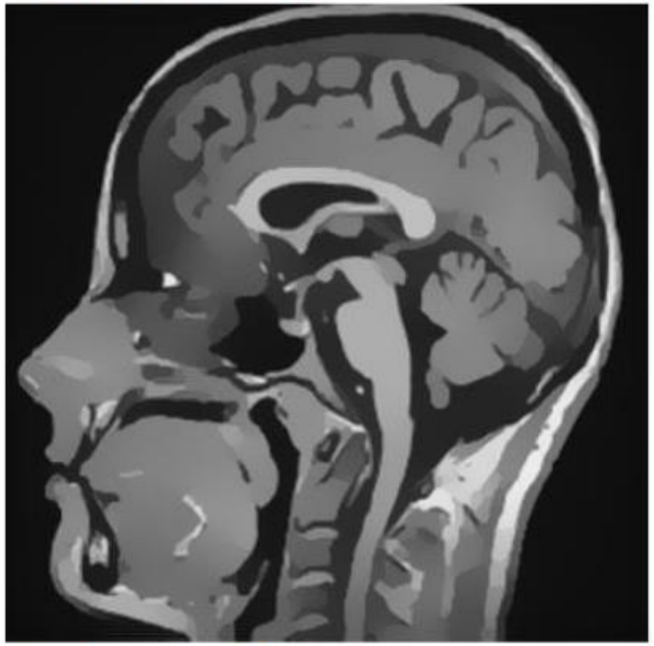
original

non-edge-preserving



gauss

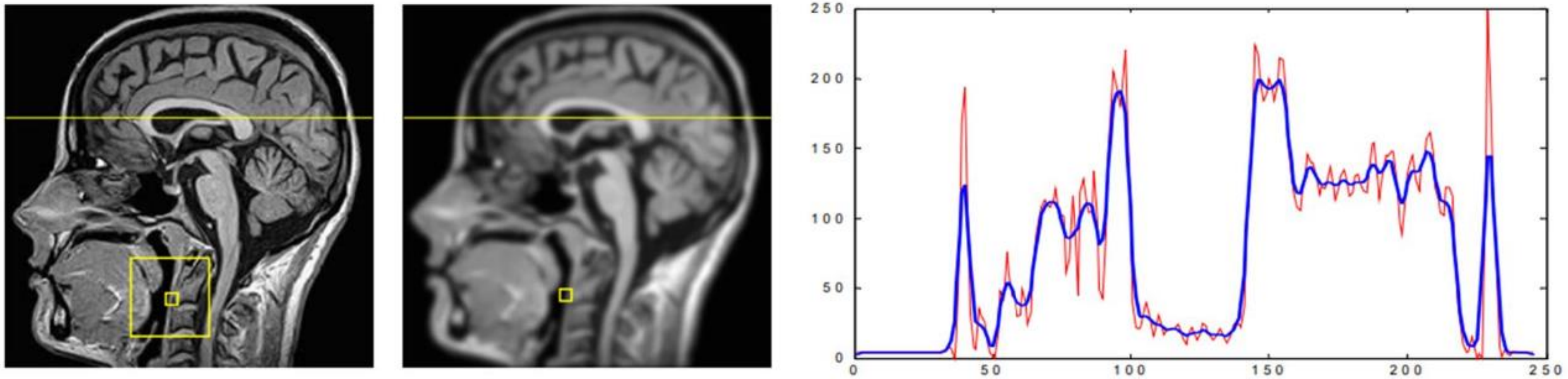
edge-preserving



anisotrop diffusion

Smoothing: mean filter

Smoothing of a signal is achieved by averaging the signal values:



With the average filter all gray values within the environment U_{ij} are weighted equally. The filter-process can be described with a filter mask:

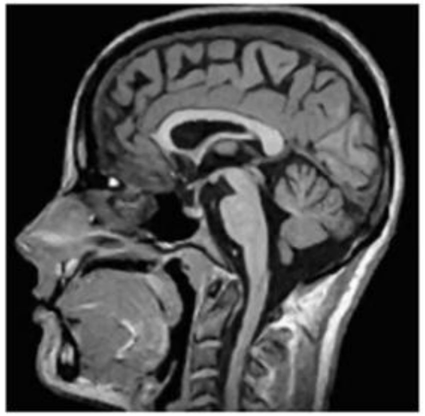
$$\mathbf{h}_A^{3x3} = \frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

$$\mathbf{h}_A^{5x5} = \frac{1}{25}\begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Smoothing: median filter

The median filter calculates the median value from the local pixel values and thus suppresses extreme values better than e.g. the arithmetic mean. It is a nonlinear filter

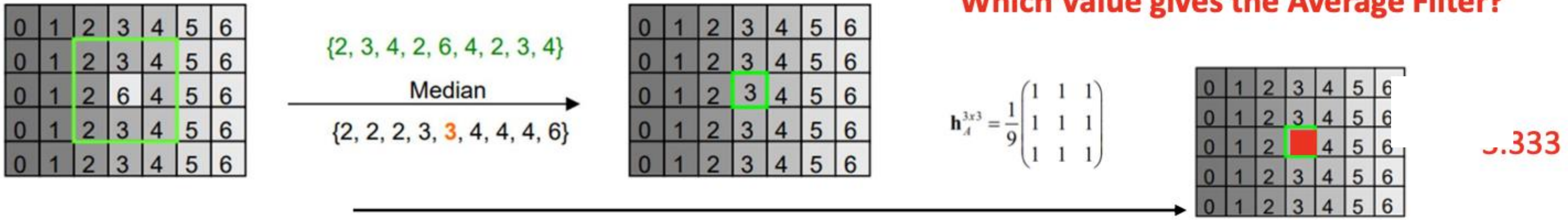
image with «Salt&Pepper»



Median-Filter

Average-Filter

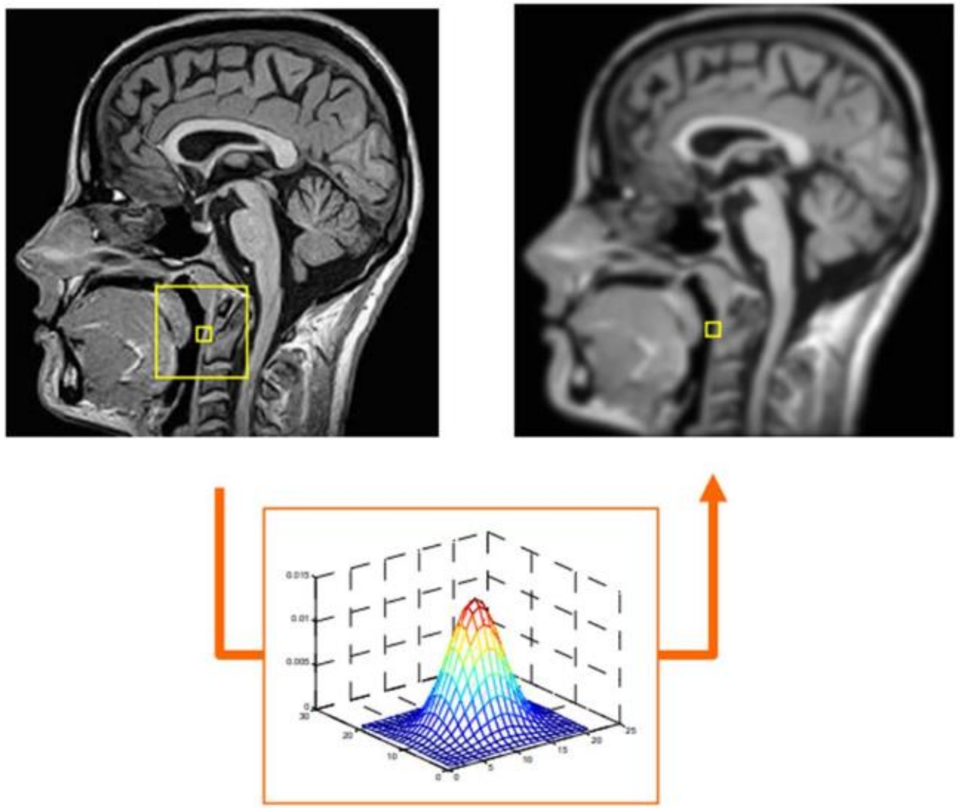
Which Value gives the Average Filter?



$$h_A^{3x3} = \frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Smoothing: Gaussian filter

With the Gaussian filter, the gray values in the environment U_{ij} are weighted according to a Gaussian function. The Gaussian function (and filter size) is determined by the std-deviation σ



For the filter size m, following should apply:

$4*\sigma + 1 \leq m \leq 6*\sigma + 1$

Examples of Gauss filter masks

$$h_G^{3\times3} = \frac{1}{32}\begin{pmatrix} 1 & 4 & 1 \\ 4 & 12 & 4 \\ 1 & 4 & 1 \end{pmatrix} \qquad h_G^{5\times5} = \frac{1}{121}\begin{pmatrix} 1 & 2 & 3 & 2 & 1 \\ 2 & 7 & 11 & 7 & 2 \\ 3 & 11 & 17 & 11 & 3 \\ 2 & 7 & 11 & 7 & 2 \\ 1 & 2 & 3 & 2 & 1 \end{pmatrix}$$

The discrete Gaussian filters can also be approximated by the binomial numbers.

The Gaussian filter is preferable to the Average filter

Smoothing: Gaussian filter

The formation of Gauss filter masks can be approximated by the binomial numbers. [then every isotropic, multidimensional Gaussian function can be considered a product of 1-dim. Gauss functions and these are described by the discrete binomial distribution, in the limite]

$$G(x,y)=\frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}=\frac{1}{\sqrt{2\pi}\,\sigma}e^{-\frac{x^2}{2\sigma^2}}\cdot\frac{1}{\sqrt{2\pi}\,\sigma}e^{-\frac{y^2}{2\sigma^2}}=G(x)\cdot G(y)$$

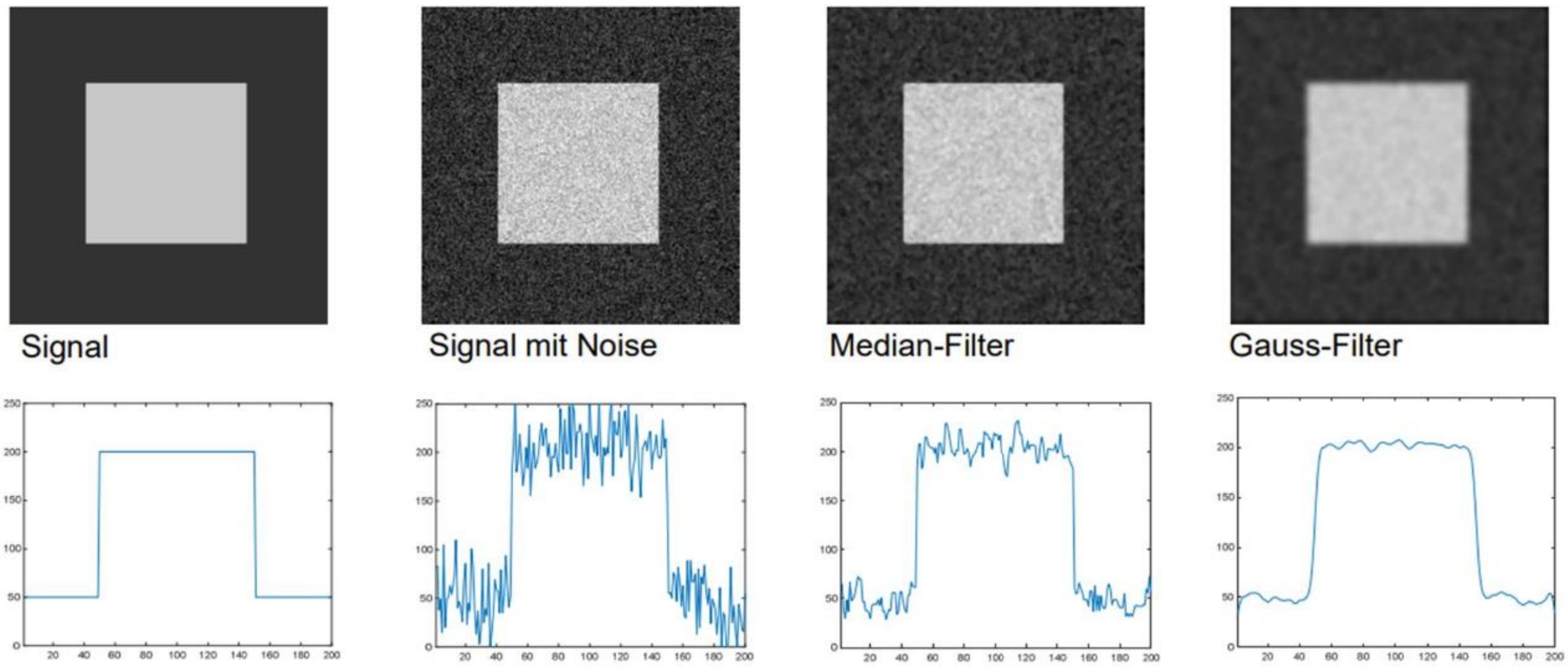
$$\begin{array}{c}
1 \\
1 \quad 1 \\
1 \quad 2 \quad 1 \\
1 \quad 3 \quad 3 \quad 1 \\
1 \quad 4 \quad 6 \quad 4 \quad 1 \\
1 \quad 5 \quad 10 \quad 10 \quad 5 \quad 1 \\
1 \quad 6 \quad 15 \quad 20 \quad 15 \quad 6 \quad 1 \\
\dots
\end{array}$$

$$h_B^{3\times3}=\frac{1}{4}\begin{pmatrix}1\\2\\1\end{pmatrix}\cdot\frac{1}{4}\begin{pmatrix}1 & 2 & 1\end{pmatrix}=\frac{1}{16}\begin{pmatrix}1 & 2 & 1\\2 & 4 & 2\\1 & 2 & 1\end{pmatrix}$$

$$h_B^{3\times5}=\frac{1}{4}\begin{pmatrix}1\\2\\1\end{pmatrix}\cdot\frac{1}{16}\begin{pmatrix}1 & 4 & 6 & 4 & 1\end{pmatrix}=\frac{1}{64}\begin{pmatrix}1 & 4 & 6 & 4 & 1\\2 & 8 & 12 & 8 & 2\\1 & 4 & 6 & 4 & 1\end{pmatrix}$$

Smoothing vs. edges

For normal-distributed noise, the median filter is less effective, the Gaussian filter is more optimal. However, smooth edges also occurs with the Gaussian filter.



Smoothing: edge-preserving filtering

To preserve the edges, adaptive smoothing does only smooth if the local variance σ_{ij} does not exceed a certain amount of noise.

image with noise $\sigma_{noise} = 36.8$

Gauss-Filter

adaptive Gauss-Filtering

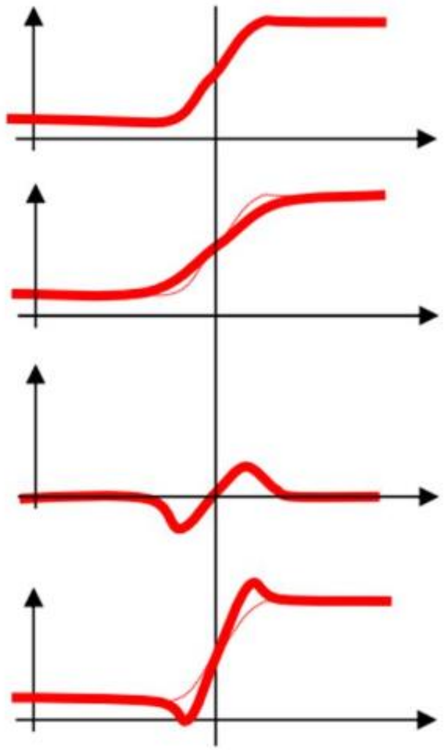
$$\tilde{b}_{ij} = \frac{\sigma_{ij}^2 - \sigma_{noise}^2}{\sigma_{ij}^2}\left(b_{ij} - \mu_{ij}\right) + \mu_{ij}$$

$\sigma_{ij} >> \sigma_{noise} \quad \Rightarrow \tilde{b}_{ij} = b_{ij}$ (no smoothing)

$\sigma_{ij} \approx \sigma_{noise} \quad \Rightarrow \tilde{b}_{ij} = \mu_{ij}$ (smoothing)

Smoothing: Unsharp masking / edge enhancement

Unsharp Masking is an approach to sharpen images: i) The smoothed image B contains the un-sharp. Ii) The difference between original A and B contains the sharpness C. iii) If the sharpness is added to the original, this results in a sharpened image



$\mathbf{A}$

$\mathbf{B} = \mathbf{h}_\sigma * \mathbf{A}$

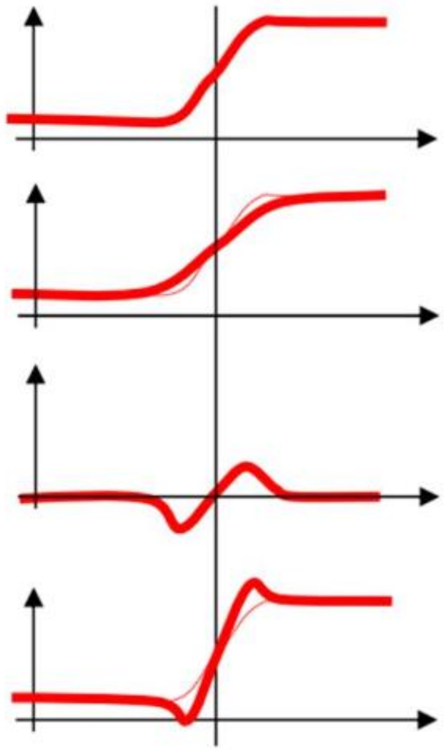
$\mathbf{C} = \mathbf{A} - \mathbf{B}$

$\widetilde{\mathbf{A}} = \mathbf{A} + \alpha \cdot \mathbf{C}$

sharped image

Smoothing: Unsharp masking / edge enhancement

Unsharp Masking is an approach to sharpen images: i) The smoothed image B contains the un-sharp. Ii) The difference between original A and B contains the sharpness C. iii) If the sharpness is added to the original, this results in a sharpened image



$\mathbf{A}$

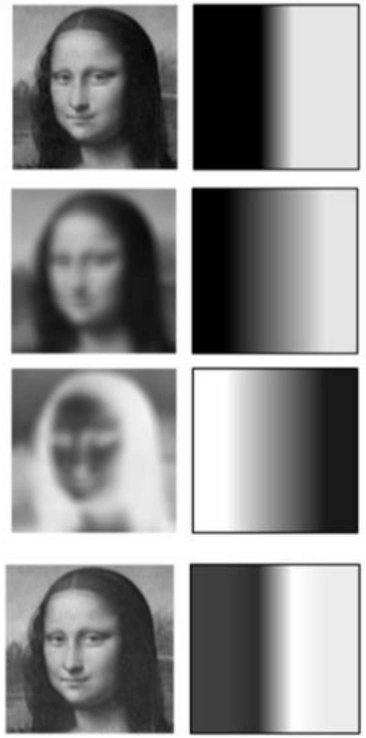
$\mathbf{B} = \mathbf{h}_\sigma * \mathbf{A}$

$\mathbf{C} = \mathbf{A} - \mathbf{B}$

$\widetilde{\mathbf{A}} = \mathbf{A} + \alpha \cdot \mathbf{C}$

sharped image

Example Detail



Smoothing: Unsharp masking / edge enhancement

- From the above operations, we can derive the corresponding kernel:

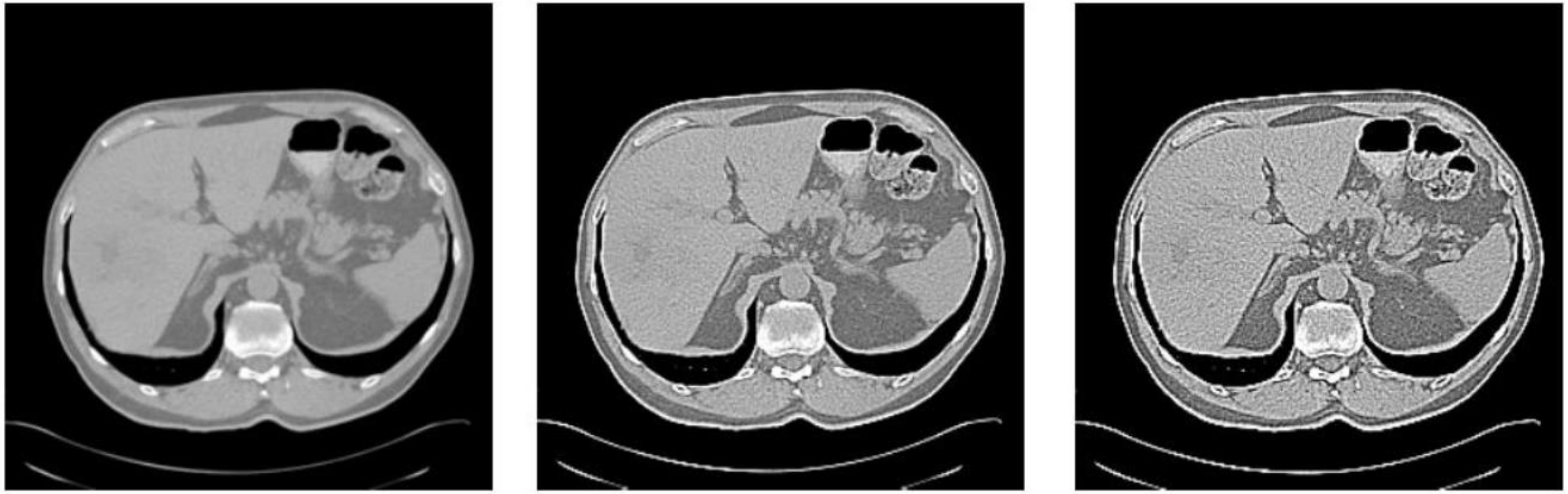
$$\mathbf{A} \to \tilde{\mathbf{A}} = \mathbf{A} + \alpha \cdot \mathbf{C} = \mathbf{A} + \alpha(\mathbf{A} - \mathbf{B}) = h_{id} * \mathbf{A} + \alpha(h_{id} * \mathbf{A} - h_{blur} * \mathbf{A}) = \underbrace{(h_{id} + \alpha(h_{id} - h_{blur}))}_{h_{sharp}} * \mathbf{A}$$

$$h_{sharp} = h_{id} + \alpha(h_{id} - h_{blur})$$

$$h_{id} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} \qquad h_{blur} = \frac{1}{16}\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

$$\Rightarrow h_{sharp} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} + \frac{\alpha}{16}\begin{pmatrix} -1 & -2 & -1 \\ -2 & 12 & -2 \\ -1 & -2 & -1 \end{pmatrix}$$

Smoothing: Unsharp masking / edge enhancement



α=2 α=4